

Session 05 – Machine Learning Modelling

Héctor J. Hortúa, PhD
2026

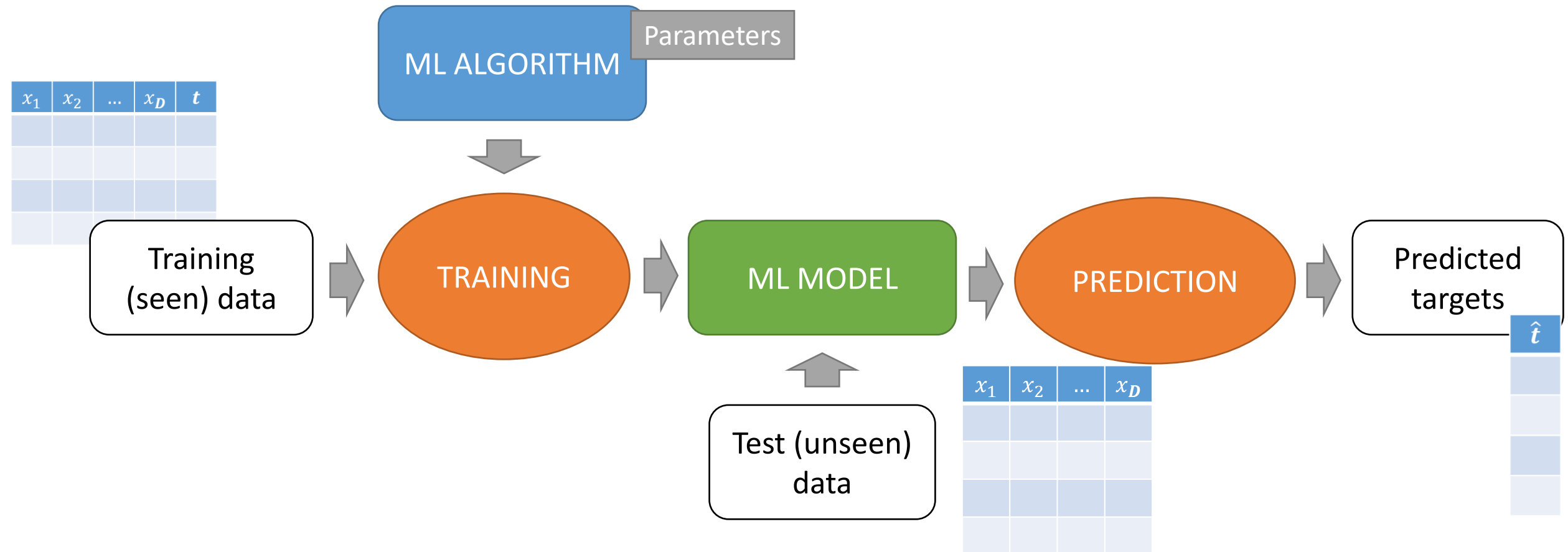
Learning outcomes

By the end of this session we should be able to:

- Describe a range of different machine learning algorithms.
- Understand the issues with the risk of overfitting and the bias/variance trade-off.
- Design and perform machine learning experiments in Python using several algorithms.

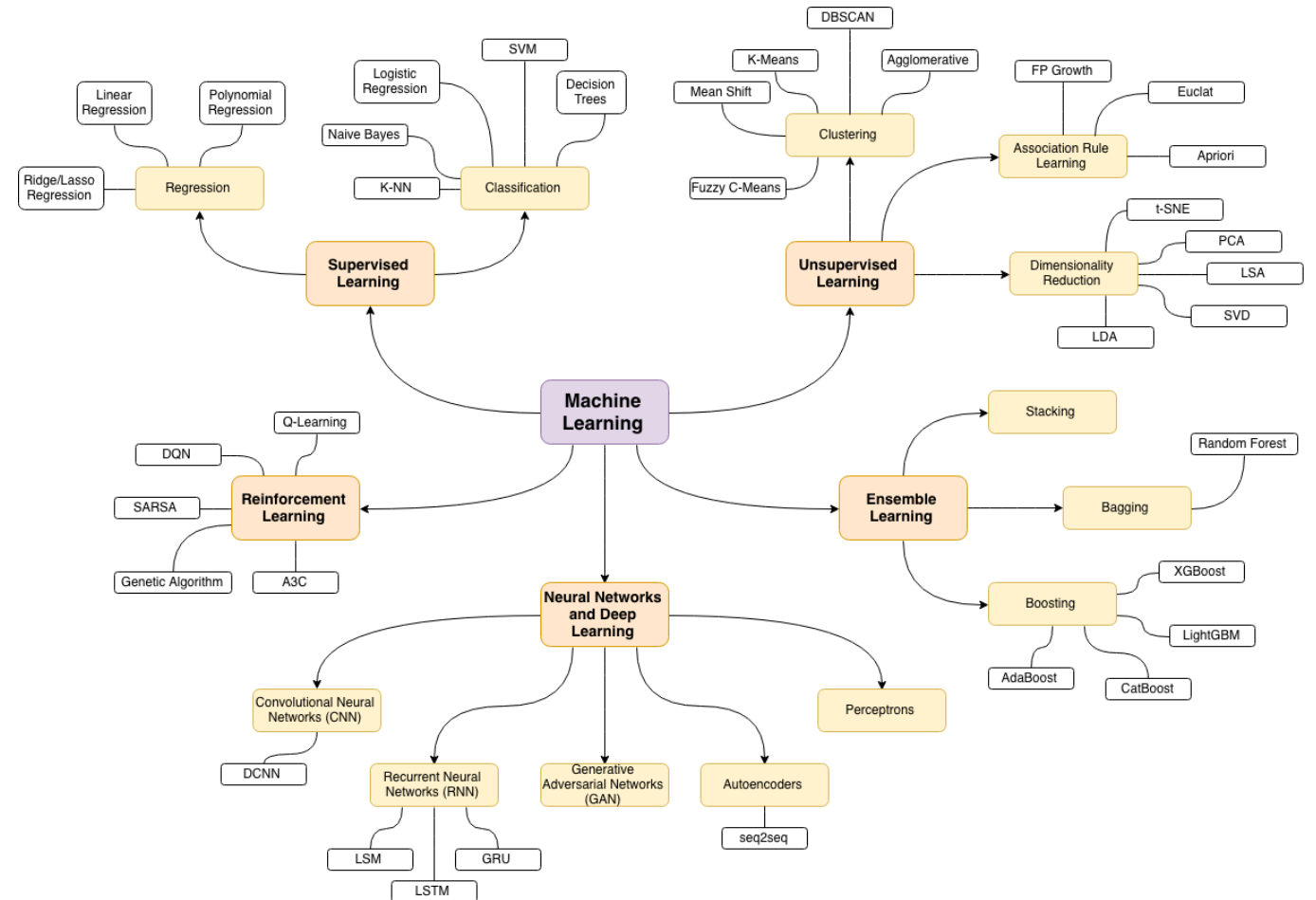
Model fitting (or training)

- A ML/Statistical model is **fitted** or trained by using data (training data) to estimate its parameters.
- In turns, a fitted model can be used to make **predictions** using **unseen** data (test data).



Machine Learning algorithms

- Logistic regression
- Artificial neural networks (ANNs)
- Support vector machines (SVMs)
- k-Nearest neighbours (kNN)
- Decision trees (CARTs)
- Ensemble learning (bagging, random forest [RF], gradient boosted machines [GBM])

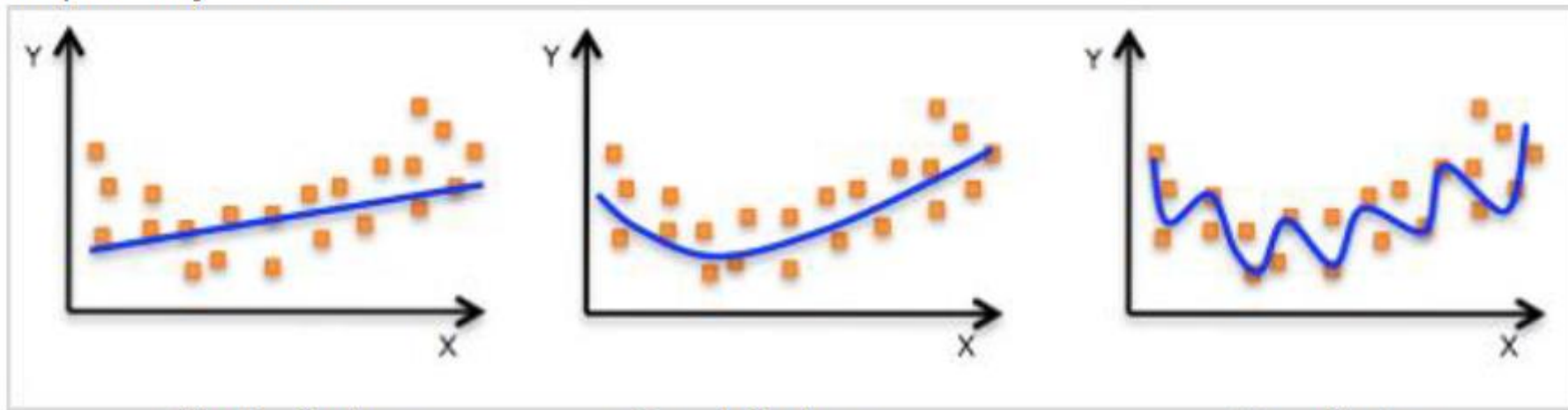


Model Validity (1)

- Generalization capability: The goal of machine learning is that the model obtained after learning should perform well on new samples, not just on samples used for training. The capability of applying a model to new samples is called generalization or robustness.
- Error: difference between the sample result predicted by the model obtained after learning and the actual sample result.
 - Training error: error that you get when you run the model on the training data.
 - Generalization error: error that you get when you run the model on new samples. Obviously, we prefer a model with a smaller generalization error.
- Underfitting: occurs when the model or the algorithm does not fit the data well enough.
- Overfitting: occurs when the training error of the model obtained after learning is small but the generalization error is large (poor generalization capability).

Model Validity (2)

- Model capacity: model's capability of fitting functions, which is also called model complexity.
 - When the capacity suits the task complexity and the amount of training data provided, the algorithm effect is usually optimal.
 - Models with insufficient capacity cannot solve complex tasks and underfitting may occur.
 - A high-capacity model can solve complex tasks, but overfitting may occur if the capacity is higher than that required by a task.



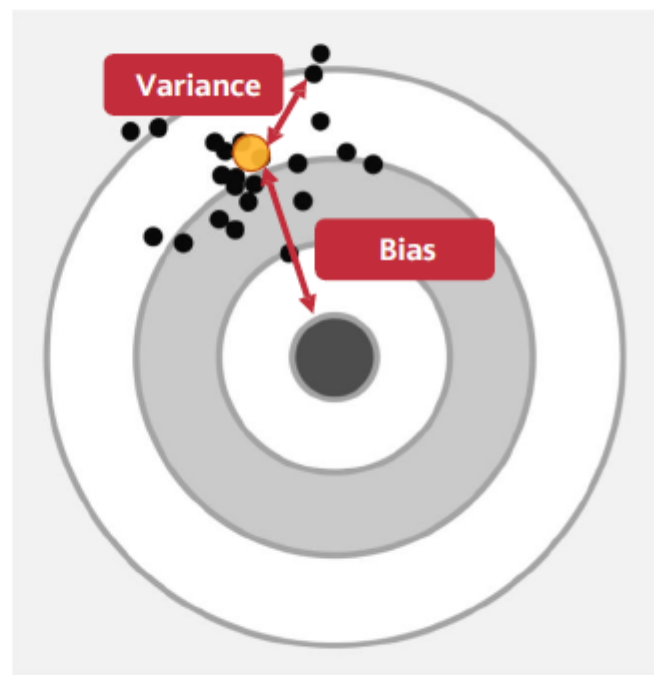
Underfitting
Not all features are learned.

Good fitting

Overfitting
Noises are learned.

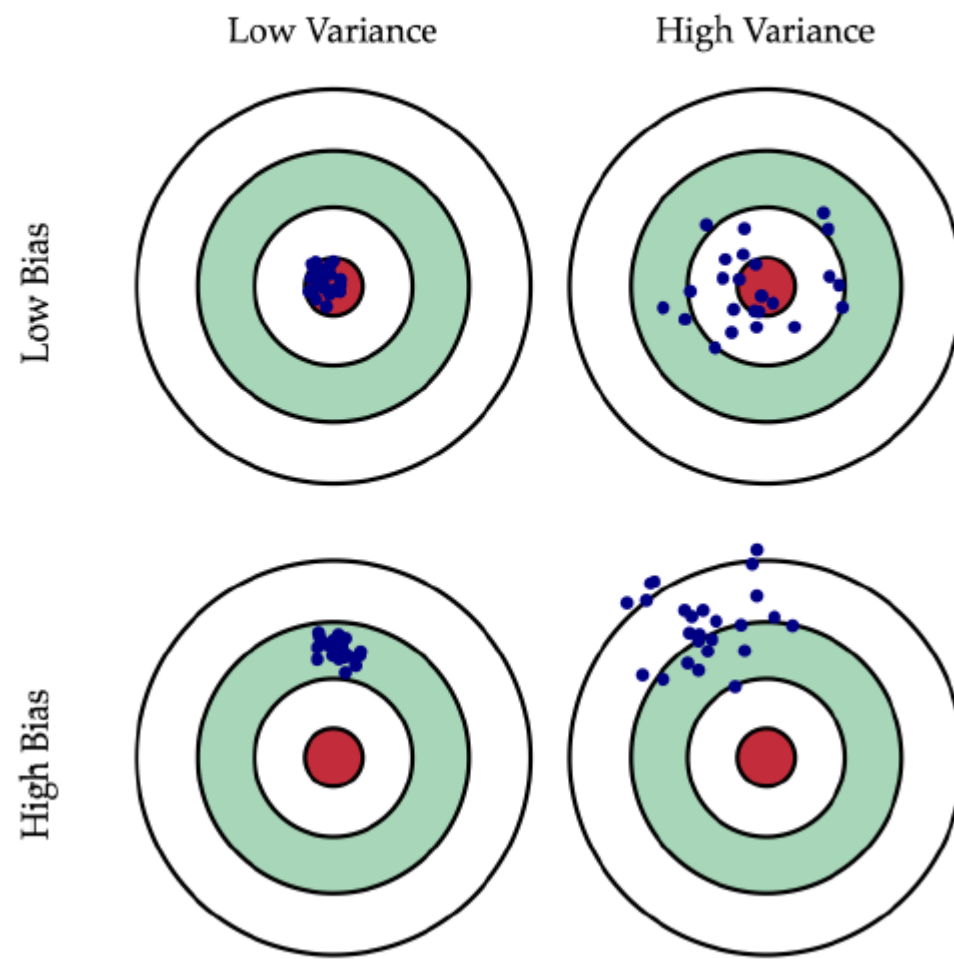
Overfitting Cause — Error

- Total error of final prediction = Bias² + Variance + Irreducible error
- Generally, the prediction error can be divided into two types:
 - Error caused by "bias"
 - Error caused by "variance"
- Variance:
 - Offset of the prediction result from the average value
 - Error caused by the model's sensitivity to small fluctuations in the training set
- Bias:
 - Difference between the expected (or average) prediction value and the correct value we are trying to predict.

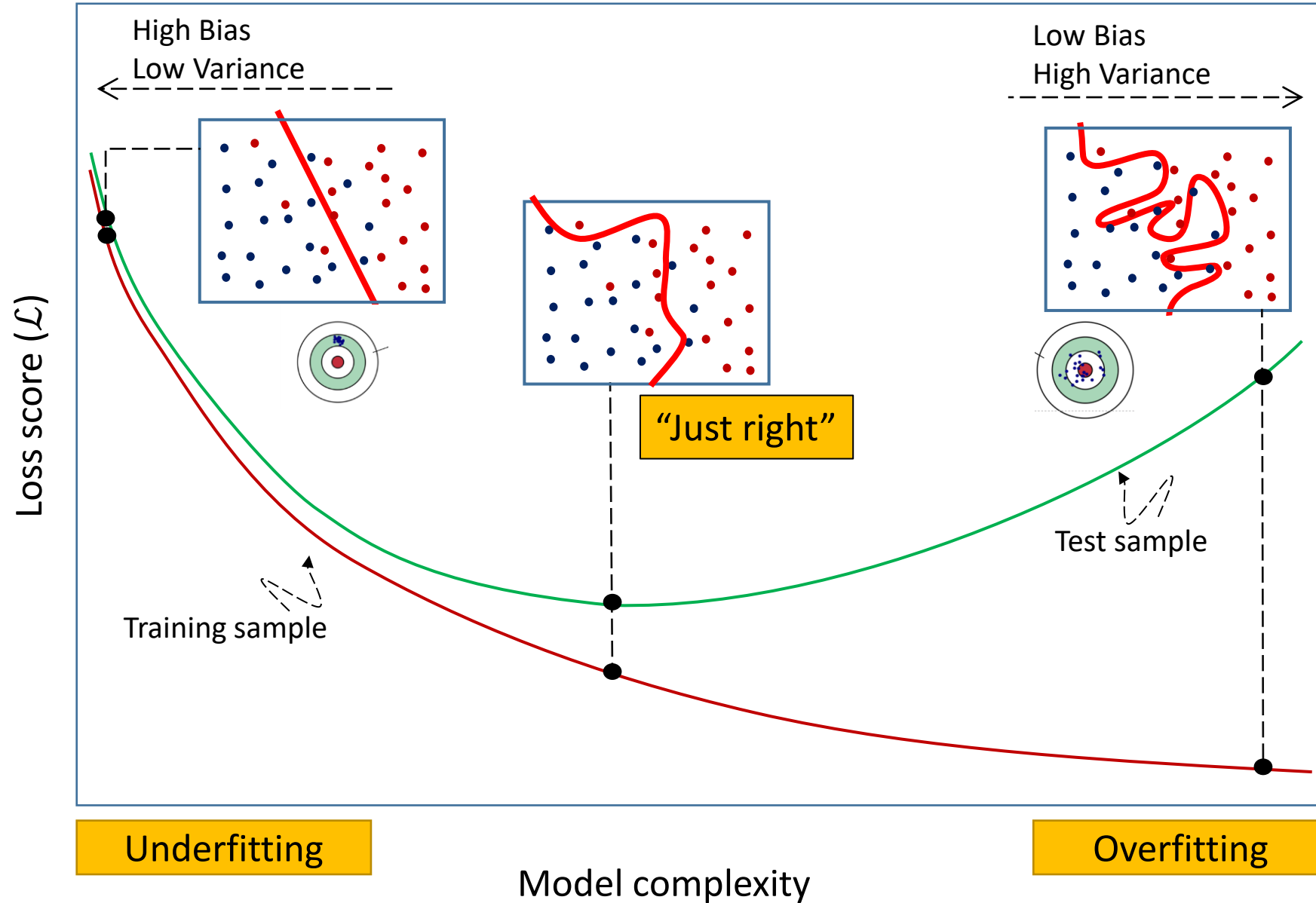


Variance and Bias

- Combinations of variance and bias are as follows:
 - Low bias & low variance → Good model
 - Low bias & high variance
 - High bias & low variance
 - High bias & high variance → Poor model
- Ideally, we want a model that can accurately capture the rules in the training data and summarize the invisible data (new data). However, it is usually impossible for the model to complete both tasks at the same time.



Bias – variance trade-off



Model selection – classification

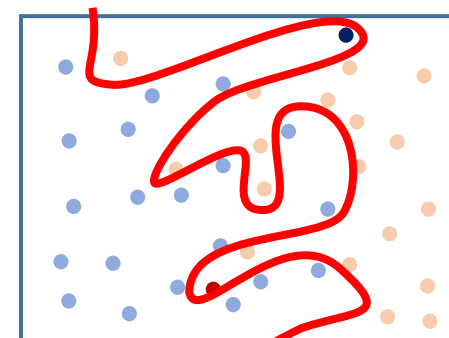
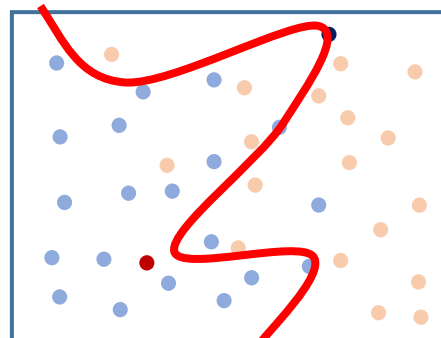
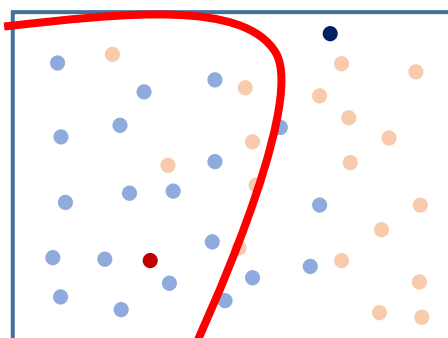
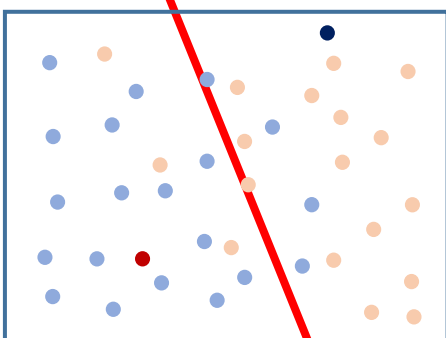
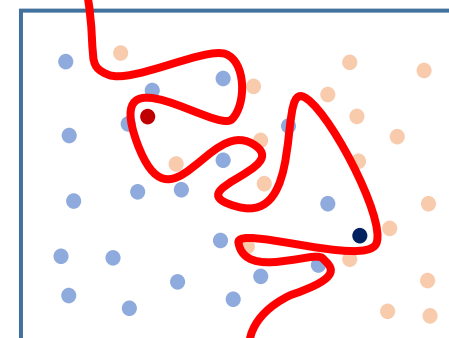
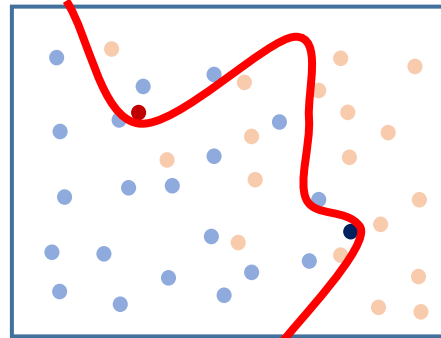
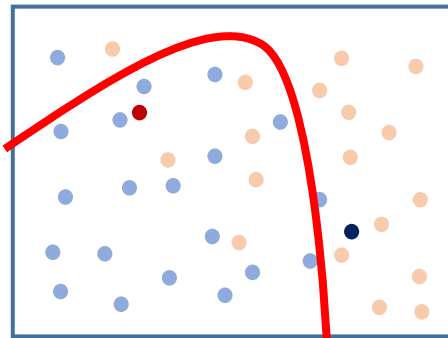
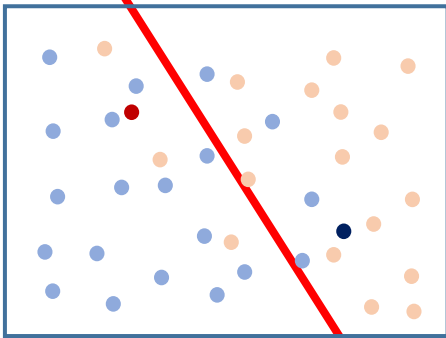
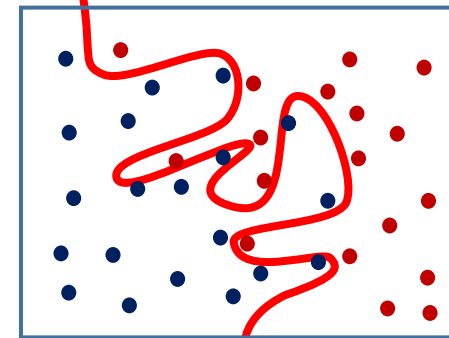
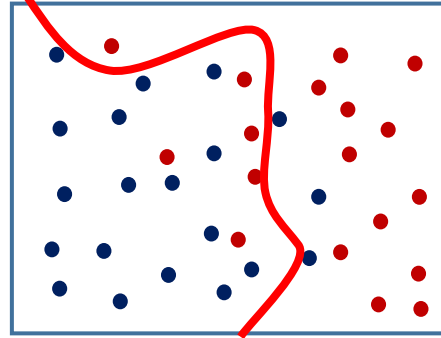
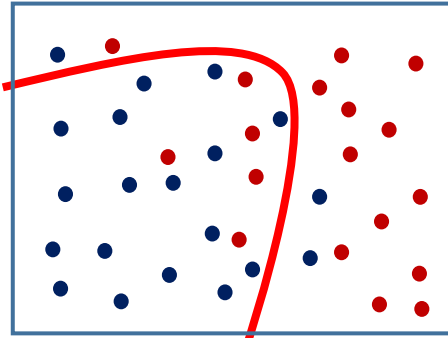
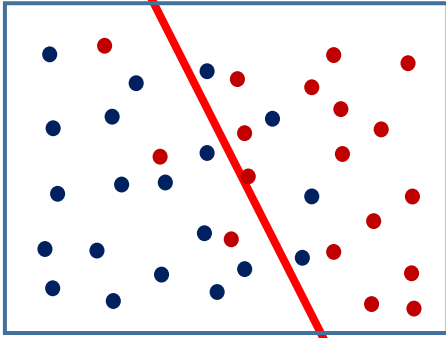
Model A

Model B

Model C

Model D

Changes in training set
↑
↓



Which one is
the right
model?

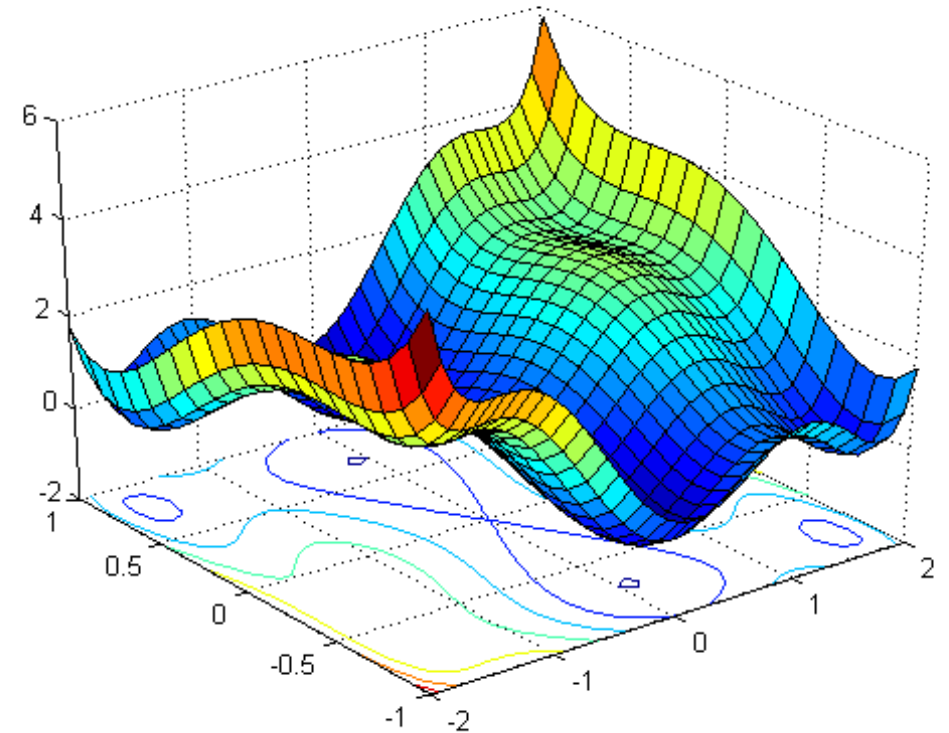
Underfitting

Increasing complexity

Overfitting

Loss function

- In ML, "loss" (also known as "cost function" or "objective function") refers to a mathematical function that quantifies the difference between the predicted outputs of a model and the actual target outputs.
- The goal of any machine learning model is to minimise the loss function.
- The loss function is used to evaluate the performance of the model during training, and the optimization algorithm tries to adjust the model's parameters so that the loss is minimised.
- The choice of the loss function depends on the problem and the type of model you are using.



Examples of loss functions

Loss	Formula	Algorithms
Residual Sum of Squares (RSS)	$\mathcal{L} = RSS = \sum_i (t_i - y_i)^2$	Regression methods (e.g. linear regression, neural networks, etc)
Mean Squared Error (MSE)	$\mathcal{L} = \frac{RSS}{n}$	Regression methods (e.g. linear regression, neural networks, etc)
Binary Cross-Entropy	$\mathcal{L} = - \sum_i t_i \log y_i + (1 - t_i) \log(1 - y_i)$	Logistic regression, 2-class neural networks
Cross-Entropy	$\mathcal{L} = - \sum_i \sum_{c=1}^C t_{ic} \log y_{ic}$	Multinomial regression, multi-class neural networks
Hinge loss	$\mathcal{L} = \sum_i \max(0, 1 - t_i y_i)$	SVMs
Gini index	$\mathcal{L} = \sum_{m=1}^M \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk})$	Decision trees, random forests.

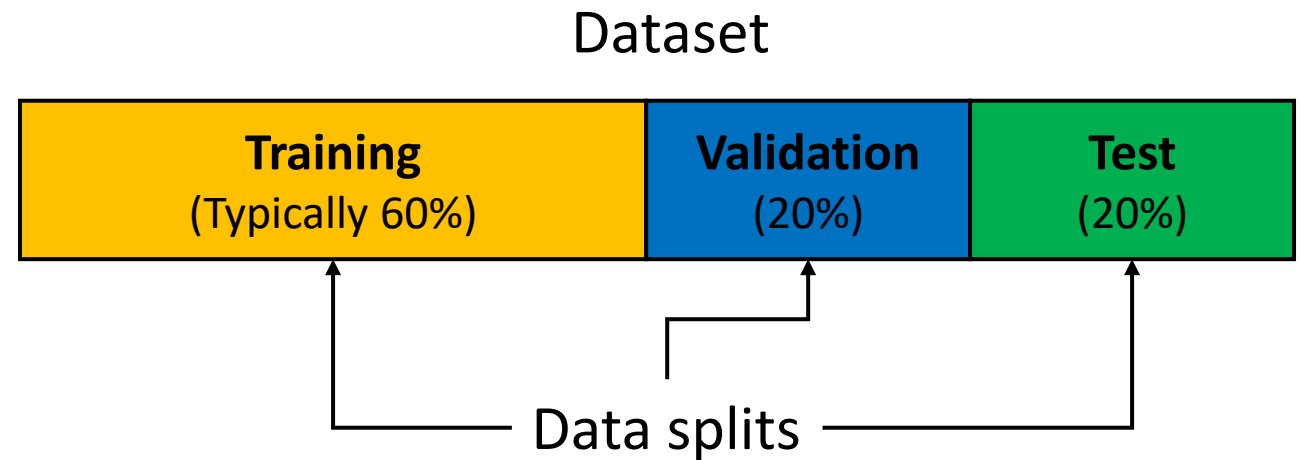
Model complexity

ML modelling deals with two separate problems:

- **Model selection** (*Optimisation*):
 - estimating the performance of different models in order to choose the best one.
- **Model assessment** (*Generalisation*):
 - having chosen a final model, estimating its prediction error (generalisation error) on a new data

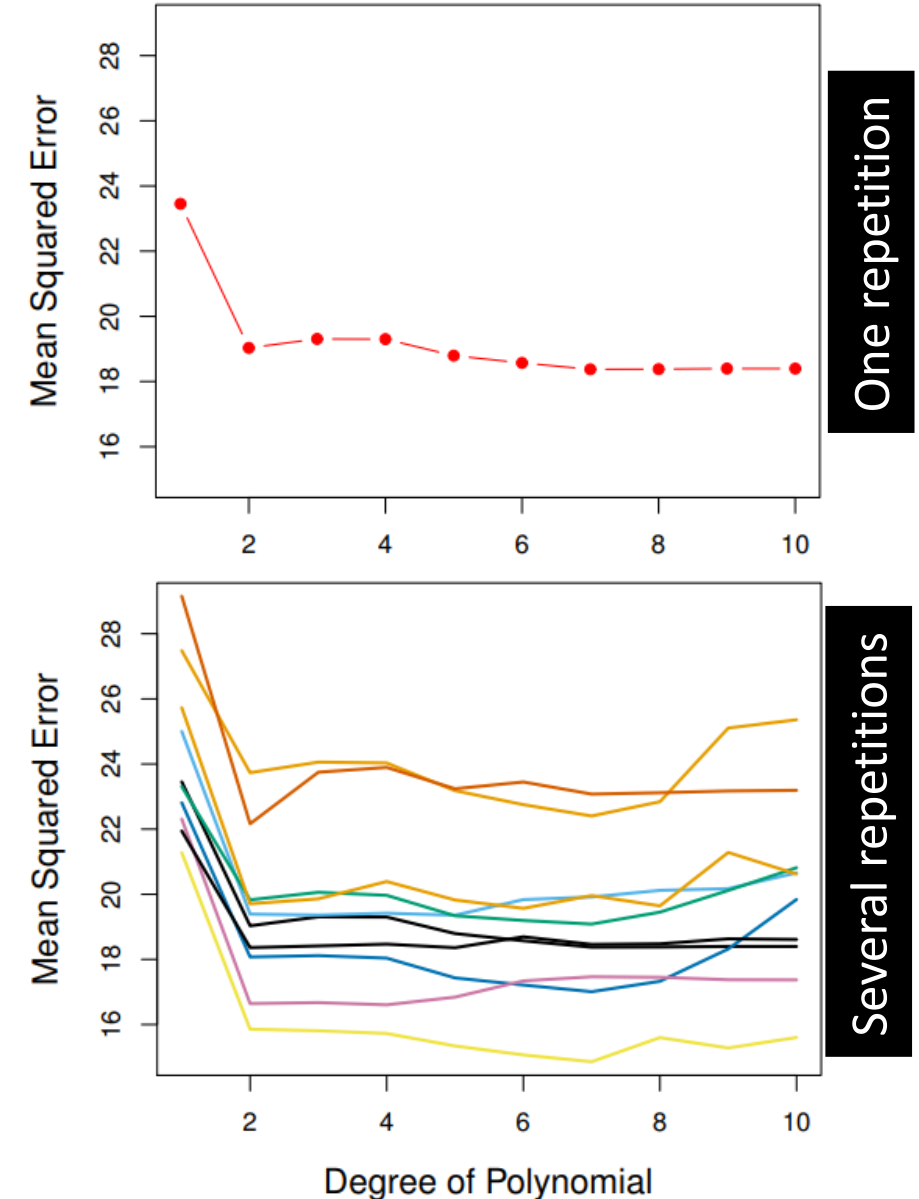
Out-of-sample resampling approach:

- **Randomly** split the dataset into training, validation and test subsets.
- **Only** the training subset is used for **training**.
- Validation split is used for model **optimisation**
- Test split is used for model **generalisation**.



Drawbacks of out-of-sample approach

- The validation estimate of the test error can be **highly** variable (depends on which observations are included in the training set and which in the validation set.)
- In the validation approach, only a subset of the observations — those that are included in the training set rather than in the validation set — are used to fit the model.
- This suggests that the validation set error may tend to **overestimate** the test error for the model fit on the entire data set.



Cross Validation (1)

- **Cross validation:** It is a statistical analysis method used to validate the performance of a classifier. The basic idea is to divide the original dataset into two parts: training set and validation set. Train the classifier using the training set and test the model using the validation set to check the classifier performance.
- **k-fold cross validation ($K - CV$):**
 - Divide the raw data into k groups (generally, evenly divided).
 - Use each subset as a validation set, and use the other $k - 1$ subsets as the training set. A total of k models can be obtained.
 - Use the mean classification accuracy of the final validation sets of k models as the performance indicator of the $K - CV$ classifier.

k -fold cross-validation

- one round involves **partitioning** a dataset into **complementary** subsets, performing the training on one subset, and validating the model on the other

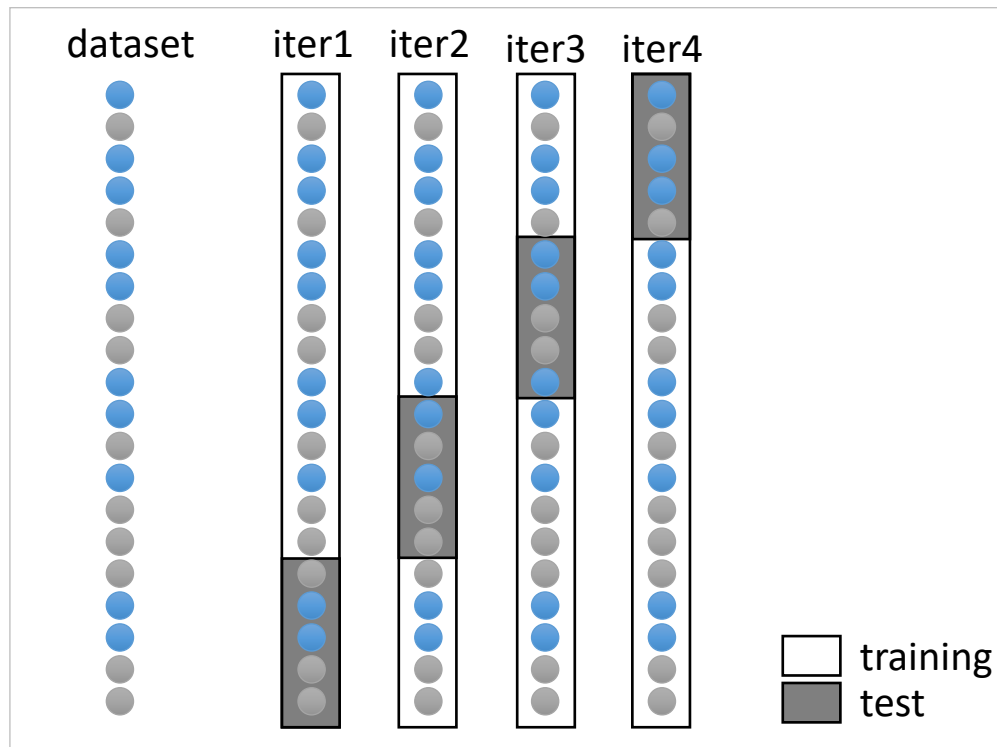


Diagram of k -fold cross-validation with $k=4$.

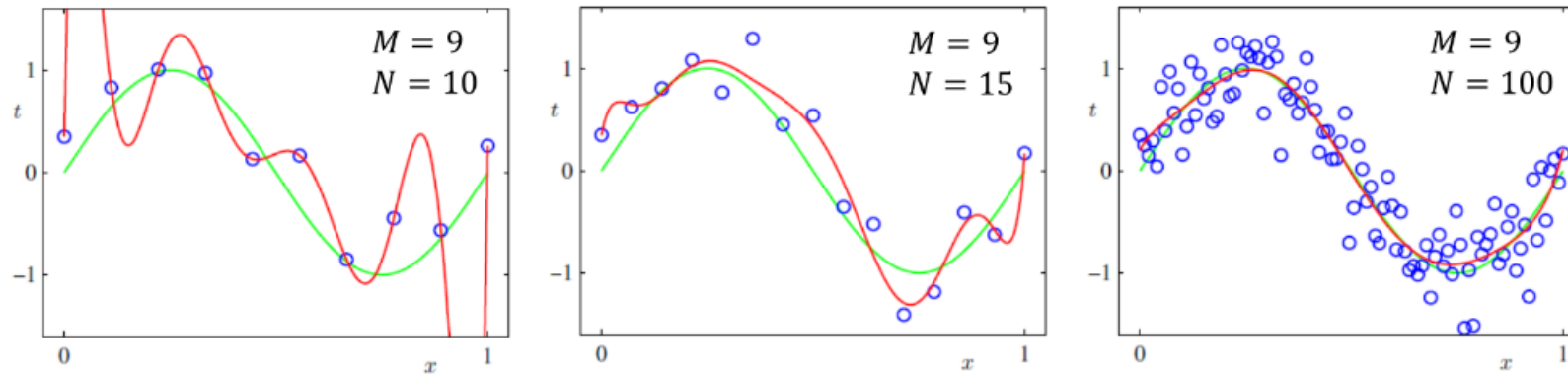
	iter1	iter2	iter3	iter4
✓	4	5	3	4
✗	1	0	2	1
Acc.	80%	100%	60%	80%

Overall accuracy = 80%
with std = 14%

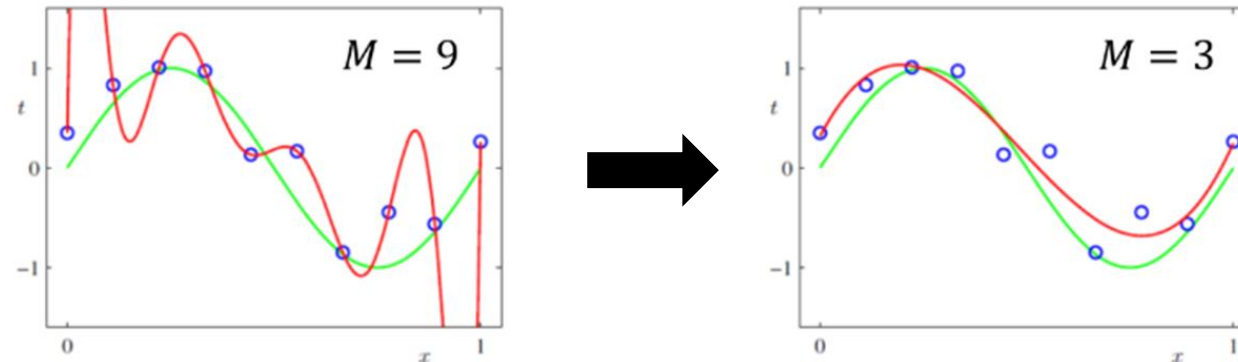
Reducing complexity in models

- In order to reduce the risk of overfitting there are two things we could do:

1. To get **more data** (which could be difficult, depending on the application)



2. To **reduce** model complexity. This means reducing the number of model parameters as much as possible.

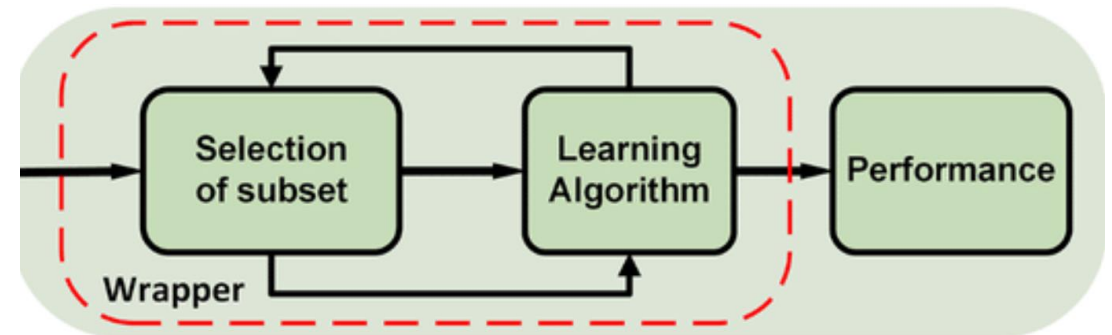
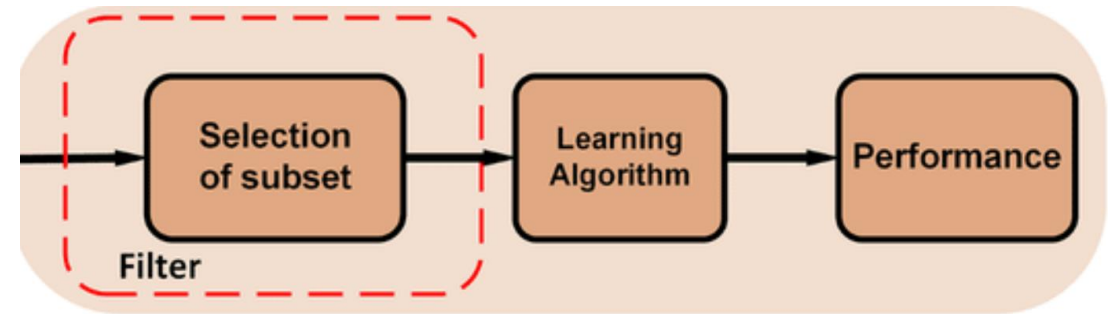


Reducing complexity via feature selection

- Automatic feature selection techniques simplify ML models by removing variables that might have little contribution to model performance.

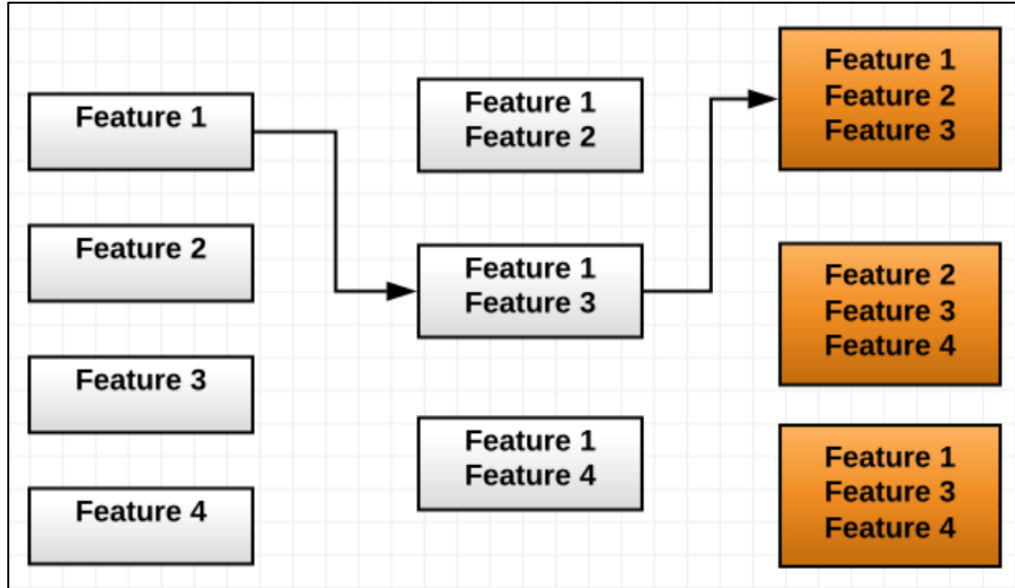
Roughly, there are two feature selection families:

- Filter methods** – the selection of features is **independent** of the ML algorithm. Selection is based on the basis of their scores in various statistical tests (χ^2 , correlation, anova, etc)
- Wrapper methods** – systematically select a subset of features and train a ML model using them (using the same or different ML algorithm). Final subset of features is chosen based on model **optimisation performance**.



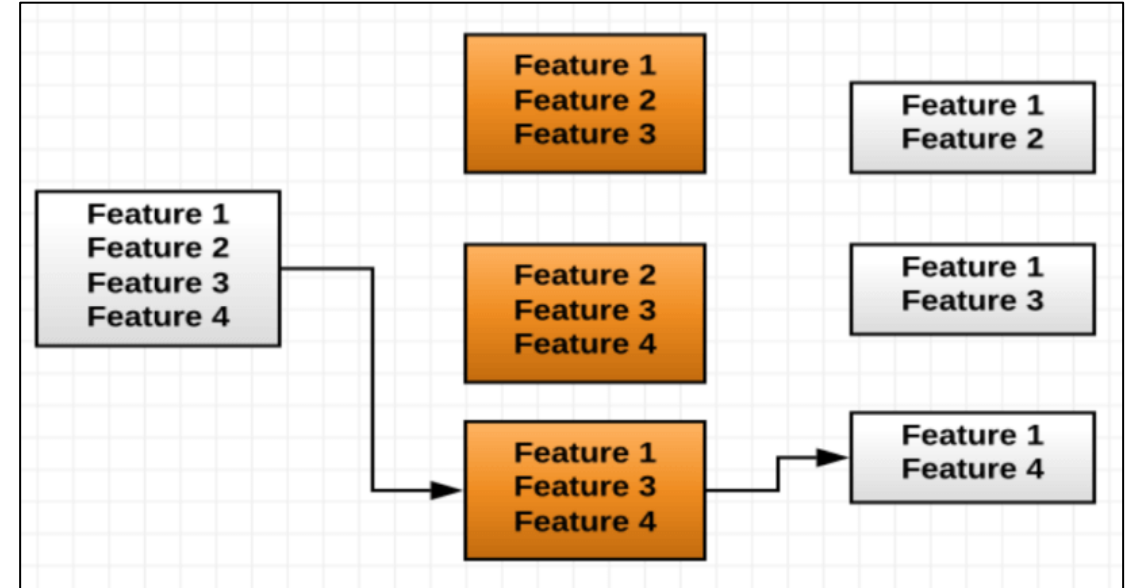
Stepwise feature selection in wrapper methods

Forward Feature Selection



1. Begin with training a model per variable.
2. Choose the one with the **lowest** loss.
3. Add a new variable to the chosen model and re-train.
4. Repeat 2-3 until exhausting all variables or until some **stopping** rule is satisfied.
5. Choose the variable subset with overall best model optimisation performance.

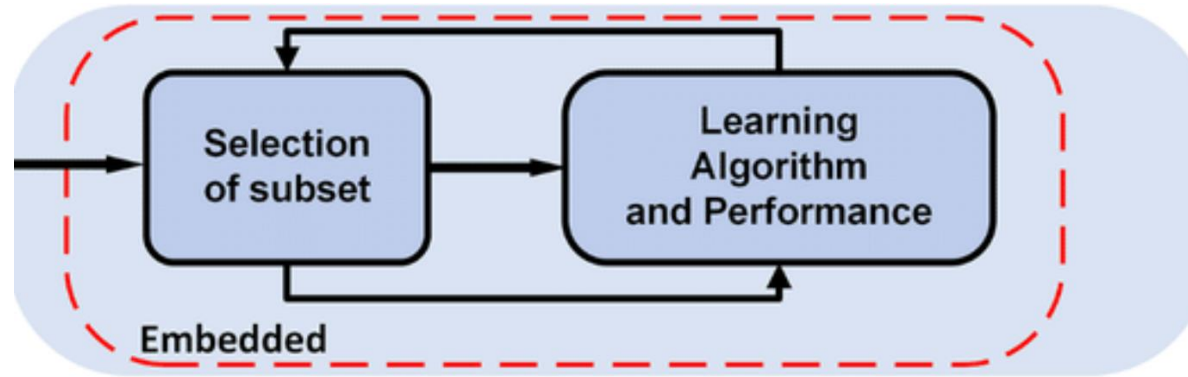
Backward Feature Selection



1. Begin with training a model using all the variables.
2. Choose the one with the **lowest** loss.
3. Remove a variable from the chosen model and re-train.
4. Repeat 2-3 until removing all variables or until some **stopping** rule is satisfied.
5. Choose the variable subset with overall best model optimisation performance.

Feature selection – Embedded methods

- **Embedded methods** – some algorithms have their own built-in feature selection methods. They don't necessarily remove features but at least control their impact on model performance.



- **Penalisation terms** – add a penalisation term to the loss function, which is typically based on model parameter values
 - $\mathcal{L} = \mathcal{L}_{DATA} + \mathcal{L}_{PEN}$
 - Commonly used in linear models (ridge regression and lasso), decision trees (pruning), and neural networks (L_1 and L_2 regularisations and weight decay).
- **Variable importance** – Tree-based methods such as Random Forest and Gradient Boosting Machines use.

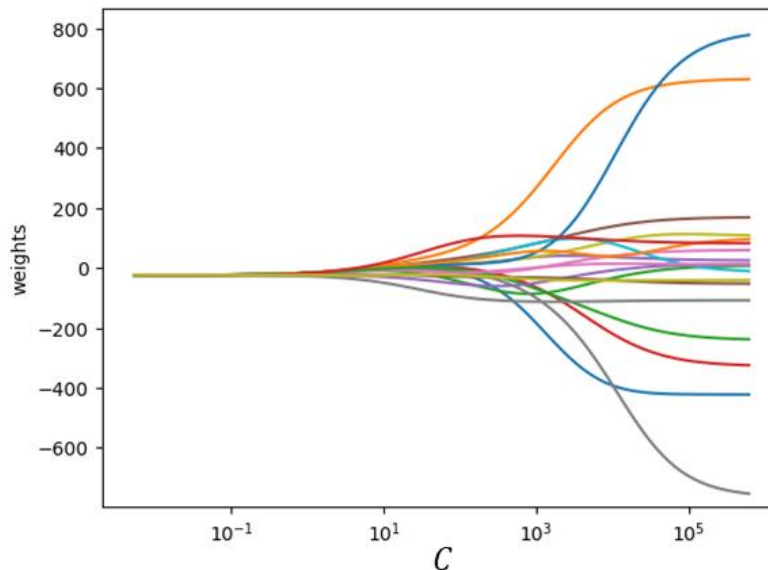
Ridge penalisation and Lasso

- Two types of penalisation terms are very popular in **linear** models:

$$\hat{\beta}^{pen} = \underset{\beta}{\operatorname{argmin}} \{ C \times \mathcal{L}(\mathbf{X}) + \mathcal{L}(\boldsymbol{\beta}) \}$$

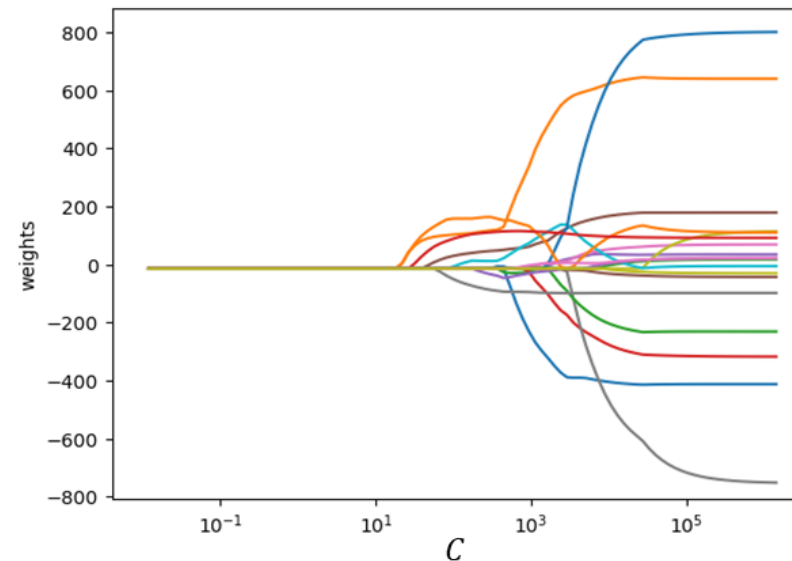
Ridge penalisation (or L_2) – **reduces** the effect of irrelevant variables.

$$\mathcal{L}(\boldsymbol{\beta}) = \frac{1}{2} \sum_{j=1}^p \beta_j^2$$



Lasso (or L_1) – **drops** irrelevant variables from the model.

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{j=1}^p |\beta_j|$$



$$\beta \rightarrow 0 \text{ as } C \rightarrow 0$$

Hyperparameters

- Hyperparameters are parameters that are **fixed before training (model assumptions)**.

Linear/Logistic Regression:

Ridge/Lasso?

Complexity parameter (C)

Decision trees:

Splitting criterion: Gini, Entropy?

Depth: maximum tree depth

Min number of samples to split

Min number of samples per leaf

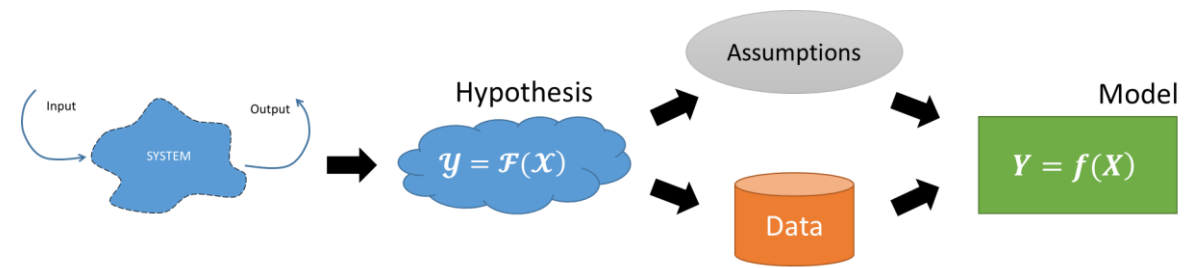
Complexity parameter

Random forest:

Similar hyperparameters as decision trees +

Number of trees

Max number of features in each split



Gradient boosting machines:

Loss: deviance (probabilistic), exponential, etc

Learning rate

Number of estimators (number of boosting stages)

Splitting criterion: Friedman MSE, MSE, MAE, etc

Others similar to RFs and DTs.

Support vector machines:

Complexity parameter (C)

Kernel function (linear, RBF, polynomial,...)

Kernel function parameters (RBF's width, degree,...)

Artificial neural networks:

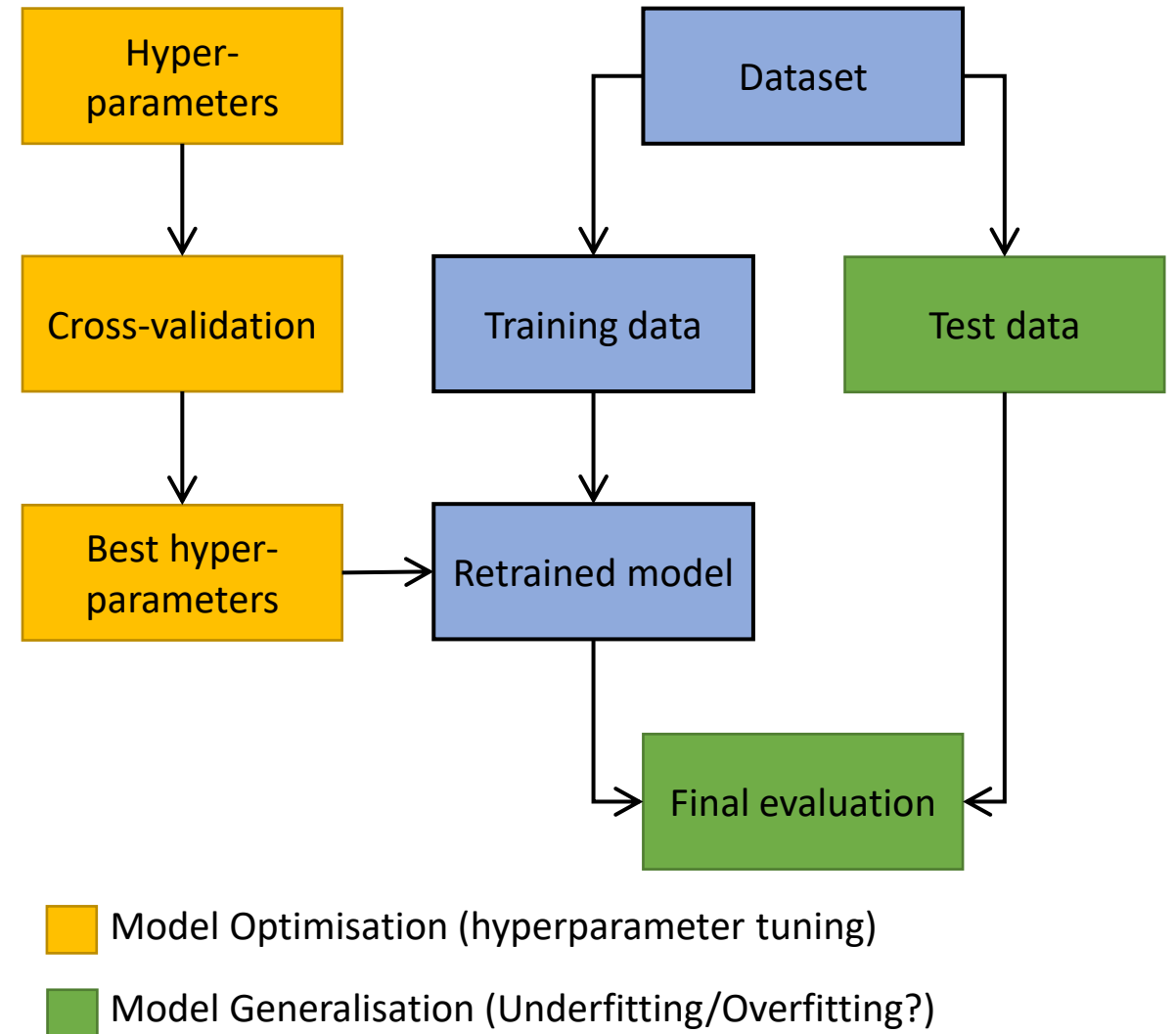
Learning rate

hidden layers and # neurons/layer

Activation functions

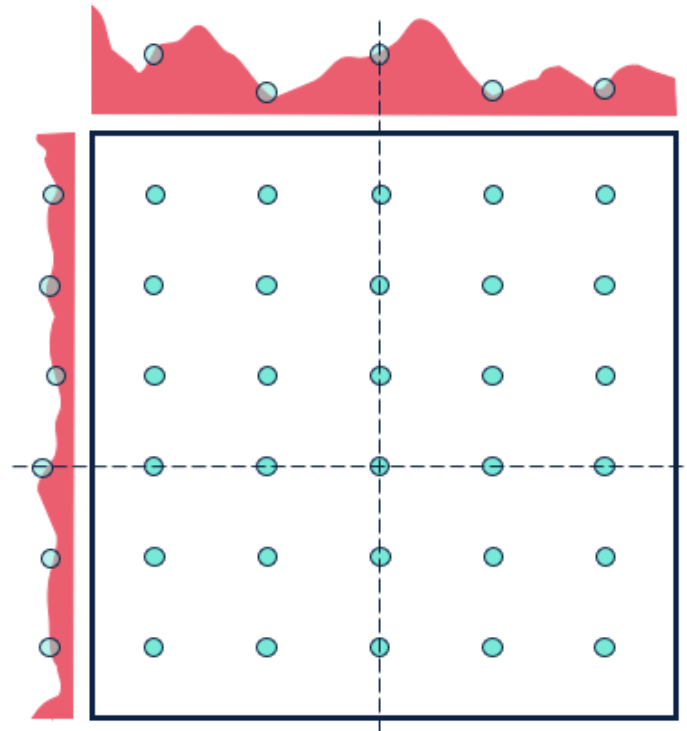
Model Optimisation and Generalisation

- It is highly recommended to test several hyperparameter set combinations.
- This gives us a typical pipeline for ML model design:
 1. Split dataset into training and test subsets.
 2. Choose the best hyperparameter set using the training subset and cross-validation [**OPTIMISATION**].
 3. Retrain the model using the chosen hyperparameter set and the full training set.
 4. Use the test subset for final evaluation [**GENERALISATION**].
- Optimisation – To find best set of hyperparameters.
- Generalisation – To test for generalisation ability (overfitting? Underfitting?)

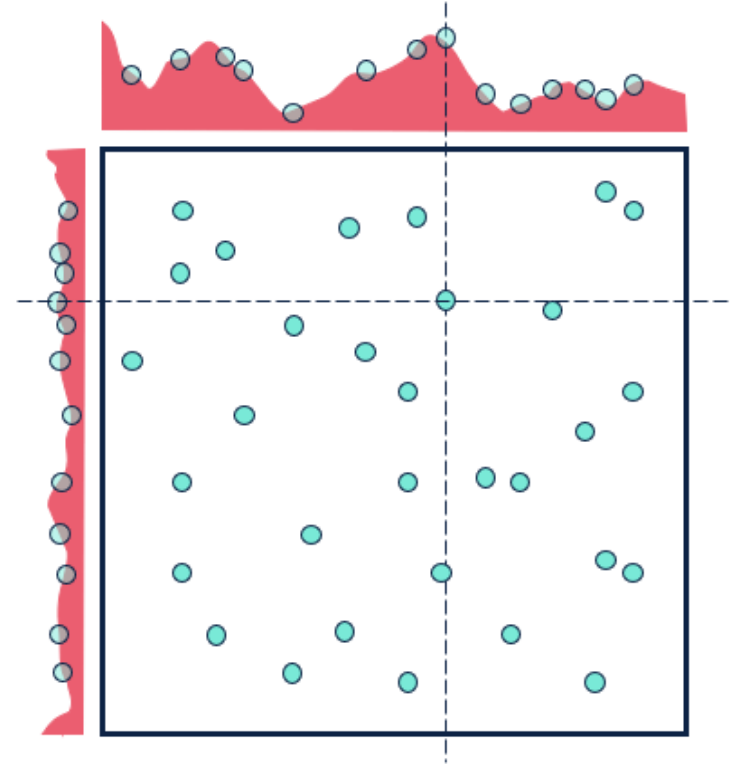


Hyperparameter search

- We should also decide on the search strategy of hyperparameters. The most common are *Grid Search* and *Random Search*.
- Grid search – the hyperparameter space is explored systematically. It is a good approach when the number of hyperparameters is small.
- Random search – Combinations of hyperparameters are tested randomly. Typically works better, particularly for large number of iterations.



Grid Search



Random Search

Designing ML experiments

- We start with a problem or task to be solved using **data** (and ML)
- Do we have the **right** data to solve the task?
- Gather a **selection** of ML algorithms
- Decide which **hyperparameters/algorithm** are to be set in advance and which will be **tuned**.
- Decide hyperparameter **search** strategy (e.g. grid or random search)
- Decide how to split the data (e.g. CV + out-of-sample, nested CV, training/validation/test splits)
- Check what else should be included (e.g. missing value imputation, normalisation, etc)